

# The training loop and the machine it runs on

*Five lines of Python, sixteen bytes per parameter, and the hardware ladder every later chapter climbs*

You launch your first real training job — a 1B-parameter model, nothing exotic — and the machine teaches you its first lesson before a single token is processed: the job won't start on your 16 GB GPU. The model is 2 GB of weights in bf16. Where did the other 14 GB go? Sixteen bytes per parameter, it turns out, is the going rate for *trainable* weights in the standard recipe — 2 for the working copy, 2 for its gradient, and 12 more for the optimizer's bookkeeping — so your 2 GB model is a 16 GB tenant, activations not included. Then the second lesson, minutes after it does start on a bigger card: `nvidia-smi` says the GPU is busy, but a profiler says it computes in bursts and starves in between, waiting on your data loader. Nothing is broken. You are simply meeting the two protagonists of the rest of this book: a training loop that is almost embarrassingly simple, and a machine whose memory system, not its arithmetic, decides how fast that loop runs.

Chapter 1 built the loop conceptually — forward, loss, backward, update — and Chapters 2 and 3 built the model it trains. This chapter puts the loop on real hardware and names everything it touches: the schedule that shapes the learning rate, the checkpoints that make failure survivable, the memory ladder from on-chip SRAM to host RAM, the interconnect that makes multiple GPUs act as one, and the exact anatomy of those sixteen bytes. Nothing here is optional knowledge: every efficiency technique in Parts II–V is an attack on a specific line item this chapter puts on the table.

## 4.1 The loop, for real this time

Here is the entire program, with the parts Chapter 1's toy version omitted — batching, device movement, mixed precision, logging — included and honest:

```
# the whole thing. Everything else in this book is making this fast.
model = Transformer(config).to("cuda")                # Ch. 3, in bf16
opt    = torch.optim.AdamW(model.parameters(), lr=3e-4,
                             betas=(0.9, 0.95), weight_decay=0.1)
sched  = warmup_cosine(opt, warmup=2000, total=max_steps) # §4.2

for step, batch in enumerate(loader):                  # packed rows (Ch. 16)
```

```

batch = batch.to("cuda", non_blocking=True)
logits = model(batch.input_ids)                # forward
loss = F.cross_entropy(logits.transpose(1, 2),  # Ch. 2's one equation
                    batch.labels)
opt.zero_grad(set_to_none=True)
loss.backward()                                # backward: ~2x forward
FLOPs
clip_grad_norm_(model.parameters(), 1.0)      # stability rail (Ch. 18)
opt.step(); sched.step()
if step % 50 == 0: log(step, loss.item())      # .item() syncs – not every
step
if step % 2000 == 0: save_checkpoint(step)     # §4.3

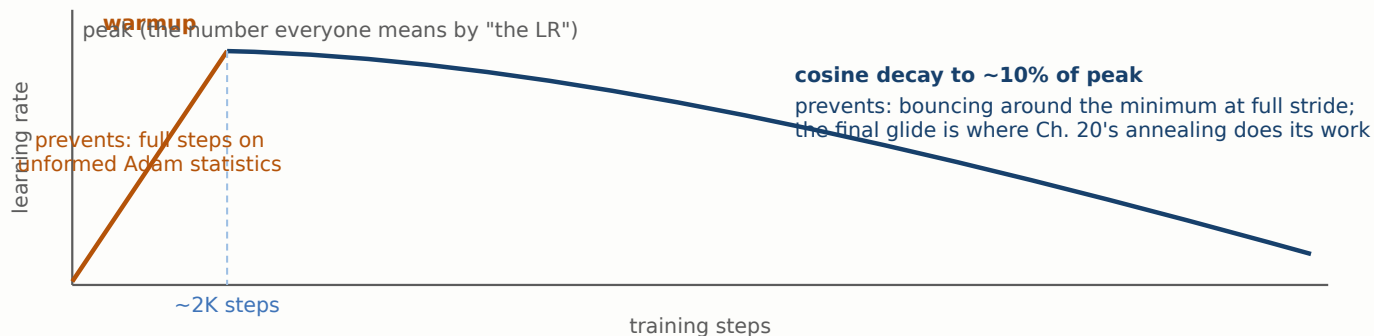
```

Read it as a materials flow, because that is how the machine experiences it. The **loader** assembles token ids on the CPU and ships them over PCIe. The **forward** pass streams every weight from GPU memory through the compute units once, writing activations as it goes. The **backward** pass streams the weights again and the activations back, producing one gradient per weight — about twice the forward's work (Chapter 1's rule, priced properly in Chapter 8). The **optimizer step** reads and writes those sixteen bytes per parameter. That's it, forever, millions of times. When people say a 70B model took months on thousands of GPUs, they mean this loop, at scale, with every line above turned into an engineering discipline: the loader becomes Chapter 10's pipelines, the forward/backward become Chapters 13–16, the step becomes Chapter 17, and the checkpoint line becomes the difference between an interruption and a catastrophe (§4.3).

## 4.2 Batches, learning rate, warmup, decay

Two knobs in that code deserve first-look intuitions now and full treatment in Chapter 17. **The batch** exists for the machine before it exists for the math: GPUs reach their throughput only when matrix multiplies are fat (§4.4), so we process hundreds of sequences per step and accept that the resulting gradient is an *average* — noisier per example, smoother per step. Bigger batches give cleaner gradients and better hardware efficiency but fewer updates per token seen; that tension, and the batch-size/learning-rate coupling it creates, is one of Chapter 17's central trades.

**The learning rate** is the single most consequential number in the config file, and it is never constant. The standard shape has two features, each existing for a specific failure. *Warmup* — a linear ramp from ~0 over the first few thousand steps — exists because Adam's denominators are statistics: the optimizer scales each parameter's update by running estimates of its gradient's magnitude,<sup>2</sup> and for the first steps those estimates are built from almost no data. Full-size steps taken on garbage statistics, applied to freshly initialized weights, is how runs blow up in minute one. *Decay* — usually a cosine glide to ~10% of peak — exists because the right step size near a minimum is smaller than the right step size far from it; Chapter 1's bowl picture, applied schedule-wide. Chapter 20 will add a twist (the annealing phase does real work in that final glide), and Chapter 17 covers the newer schedules that decouple the two, but warmup-then-cosine is the shape to recognize:



**Figure 4.1 — The standard schedule and the failure each phase prevents.** Warmup protects the fragile early steps while Adam's per-parameter statistics form; cosine decay shrinks the stride as the model closes in. The peak value between them is the most-tuned number in the field — Chapter 17 is largely about choosing it.

## 4.3 Checkpoints and resuming

At scale, hardware fails as a matter of routine: Meta's 405B run logged **419 unexpected interruptions in 54 days — one every three hours**, three quarters of them hardware.<sup>1</sup> The checkpoint line in §4.1's loop is what converts each of those from "restart the month" to "lose twenty minutes," and writing it correctly means knowing what state actually defines a run. Weights alone are not a checkpoint. A resumable checkpoint is *weights + optimizer state + schedule step + RNG states + data-loader position* — miss any one and the resumed run is a different run:

```
# a checkpoint is the run's full state, not the model's
def save_checkpoint(step):
    torch.save({
        "model": model.state_dict(),          # bf16 working weights
        "opt":   opt.state_dict(),             # fp32 master + Adam m, v (the
bulk!)
        "sched": sched.state_dict(), "step": step,
        "rng":   {"torch": torch.get_rng_state(),
                  "cuda": torch.cuda.get_rng_state_all(),
                  "numpy": np.random.get_state()},
        "loader": loader.state_dict(),        # which shard, which position
    }, f"ckpt_{step:07d}.pt")
# ~14 bytes/param on disk: 7B ≈ 98 GB, 405B ≈ 5.7 TB per checkpoint.
# Resume and verify: the next 100 loss values should match the old run's
# exactly. If they don't, some state above was lost — find it now, not at 70B.
```

Two consequences of the sizes are worth feeling. Writing ~100 GB every half hour is real storage traffic — which is why serious runs checkpoint asynchronously (copy to host RAM, write in the background) and why Chapter 14's distributed checkpointing shards the write across the cluster. And the *frequency* is arithmetic, not taste: with failures every ~3 hours, checkpointing every 30 minutes loses on average 15 minutes of work per failure — ~8% of the run — while checkpointing daily would lose half a day each time. Set the interval from your measured failure rate and checkpoint write cost; both belong on the dashboard.

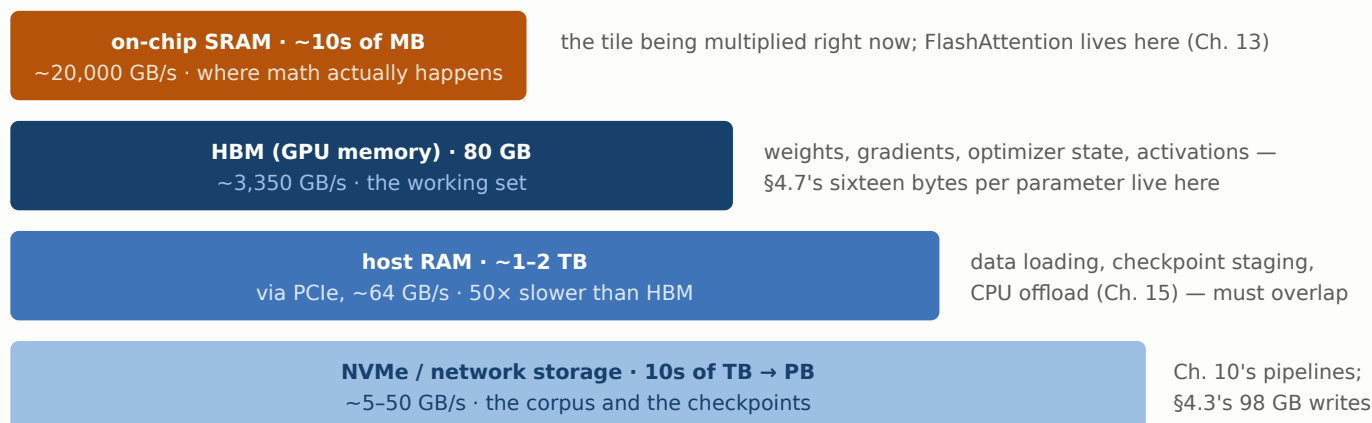
## 4.4 What a GPU is good at

A CPU is a few dozen brilliant workers; a GPU is over a hundred thousand simple ones, organized so they all execute the same instruction on different data at once. That design makes one job astonishingly cheap: **dense matrix multiplication**, the operation Chapter 3 showed the transformer is mostly made of. Modern accelerators go further and dedicate silicon to it — tensor cores that consume small matrix tiles whole — so an H100 delivers roughly **1,000 TFLOP/s in bf16 matmuls**<sup>5</sup> but only a few tens of TFLOP/s on everything else. The corollary that runs this whole book: the machine is only fast when you feed it *large, regular* matrix multiplies. Small batches, ragged shapes, elementwise-heavy code, Python in the inner loop — all of them leave the hundred thousand workers idle. When Chapter 8 defines MFU (the fraction of peak FLOP/s your training actually achieves) and calls 40–50% excellent, this is why the number isn't 100: some of every step is spent on work the matmul silicon can't help with, and — more often — on *waiting for bytes*.

## 4.5 The memory hierarchy: where the bytes live

Waiting for bytes deserves its own map, because it is the single most predictive fact about performance in this book. Memory on a training node is a ladder: each rung down holds ~10–100× more data and delivers it ~10× slower. The whole art of fast training — FlashAttention's tiling (Ch. 13), activation checkpointing (Ch. 15), fused kernels — is arranging computation so the busy data stays on high rungs:

**THE LADDER (one H100-class node) — each rung: ~10-100× the capacity, ~1/10 the speed**



**Reading the ladder is reading the book: Parts III-IV are schemes for doing more math per trip down a rung.**

**Figure 4.2 — The memory ladder.** Compute happens only at the top; data lives lower down. The 5× bandwidth gap between SRAM and HBM explains Chapter 13's tiling obsession; the 50× gap between HBM and host RAM explains why the data loader (§4.9) and checkpoint writes (§4.3) must overlap with compute or stall the run.

## 4.6 More than one GPU: why interconnect matters

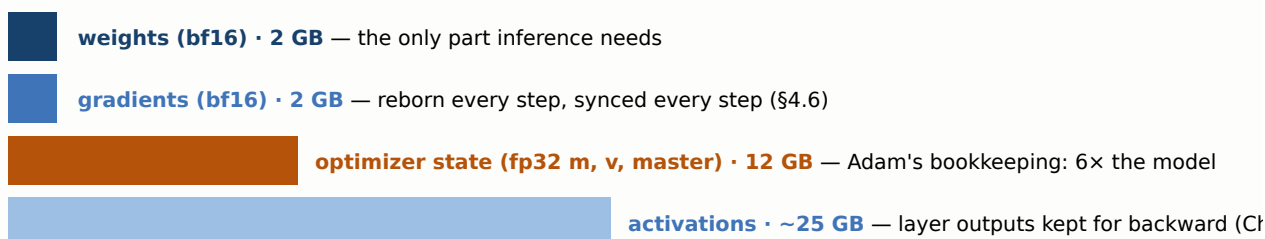
The moment training spans two GPUs, a new physical resource enters the loop: the wire between them. In the simplest multi-GPU recipe (data parallelism, Chapter 14), every GPU holds a full copy of the model, processes its own slice of

the batch, and then — every single step — all copies must *agree* on the gradient: an all-reduce that moves, for our 1B model, 2 GB of bf16 gradients per step. The wires form their own ladder: **NVLink** inside a node moves ~900 GB/s between GPUs; **InfiniBand** or **RoCE** between nodes moves ~50–100 GB/s; commodity Ethernet, far less. On the top rung the 2 GB sync disappears behind compute; across a slow rung it becomes the step time. This is why cluster topology is a training hyperparameter, why Chapter 14 spends pages on *which* parallelism crosses *which* wire, and why the ring-based algorithms there (cousins of Chapter 21's ring attention) exist at all: they turn "everyone talks to everyone" into "everyone talks to a neighbor," which is what the wires can actually do.

## 4.7 The memory anatomy: sixteen bytes per parameter, plus activations

Now the opening mystery, dissected. Training a parameter in the standard mixed-precision Adam recipe stores five things: the bf16 **working weight** the forward pass reads (2 bytes); its bf16 **gradient** (2); Adam's two fp32 **moment estimates** — the running mean and the running magnitude that give the optimizer its per-parameter step sizes (4 + 4); and an fp32 **master copy** of the weight, kept because thousands of tiny bf16 updates would round to nothing (4).<sup>3</sup> Sixteen bytes, of which the model you ship is two:

### ONE 1B-PARAMETER MODEL ON ONE GPU (batch 8 × 4,096 tokens, no tricks)



Total ≈ 41 GB for a "2 GB model" — and only the activations bar bends with your choices today: it scales with batch × sequence length, shrinks ~4× with activation checkpointing (Ch. 15), and vanishes at inference.

**The fixed 16 bytes/param are why a 70B model needs 1.1 TB of state before its first token — no single GPU holds it, which is the entire reason Chapters 14 and 15 exist.**

**Figure 4.3 — Where the GPU memory goes.** The 16 bytes/param of training state (weights, gradients, optimizer) are fixed by the recipe; activations are the elastic term. Scale the anatomy to 70B and it demands 1.1 TB — sharding that state across devices (ZeRO/FSDP,<sup>4</sup> Ch. 14) and shrinking the activation bar (Ch. 15) are the two responses.

## 4.8 The software stack, positioned

Six layers of software stand between §4.1's loop and the silicon, and knowing which layer owns which problem saves days of misdirected debugging. **PyTorch** owns tensors, autograd, and the dispatch to CUDA kernels — when a shape is wrong or a gradient is **None**, you are here. **Fused kernels** (FlashAttention, fused optimizers, `torch.compile`'s output) own the gap between naive PyTorch and the ladder of §4.5 — when MFU is low with correct code, look here. **Transformers/tokenizers** own model definitions and the text → ids boundary (Chapters 2, 11) — when special tokens or chat templates misbehave, here. **Datasets and the loader** own the corpus-to-batch pipeline (Chapters 10, 16) —

when the GPU starves, here (§4.9). **The distributed layer** (FSDP, DeepSpeed, Megatron) owns §4.6's wires and §4.7's sharding — Chapter 14. And **the harness around everything** — experiment tracking, eval runners (Chapter 9), launchers — owns reproducibility. Appendix A pins versions and configs for the whole stack as this book uses it; the mental model to keep is the ownership map, because "training is slow" is six different bugs in six different layers.

## 4.9 Practical considerations

**Debug OOMs in cost order.** Out-of-memory is the fundamentals-era rite of passage, and Figure 4.3 is the triage map: activations are the elastic term, so first reduce the per-step batch and compensate with gradient accumulation (same tokens per update, less resident memory); then turn on activation checkpointing (Ch. 15) and pay ~30% compute for a ~4× activation cut; then question sequence length (are you padding to 8K for 900-token data? — Chapter 16 says stop); only then reach for the heavy machinery of sharding and offload. What never works: shrinking the model to fit the tooling before auditing the four bars.

**Starved GPUs look busy.** The classic pathology from the opening: utilization graphs sawtooth while tokenization runs in the loop or the loader runs single-process. The fixes are boring and total: tokenize offline (once, to disk — Chapter 10), give the loader workers and pinned memory with prefetch, and overlap host-to-device copies with compute. Measure with a profiler trace, not `nvidia-smi`: five minutes of trace reading routinely buys back 30% of a step.

**Determinism is a feature you build, not assume.** Bitwise-identical resume (§4.3's verification) requires seeding everything, capturing RNG states, and accepting that some fast kernels are non-deterministic unless told otherwise. Decide your policy in week one: full determinism for debugging runs, documented non-determinism (with seeds logged) for production speed — and verify resume correctness *before* the run you can't afford to lose.

### Common pitfalls

---

#### Logging `.item()` every step.

Each call synchronizes CPU and GPU, serializing the pipeline. Log every N steps, or accumulate on-device.

---

#### Weights-only checkpoints.

Resuming without Adam's moments restarts the optimizer cold mid-schedule — a silent quality tax. Save the full state dict of §4.3.

---

#### fp16 where bf16 was meant.

fp16's narrow exponent needs loss scaling and still overflows; bf16 shares fp32's range and just works. On modern hardware, bf16 unless you have a reason.

---

#### Eval contaminating the loop.

Forgetting `model.eval()` / `torch.no_grad()` in validation burns memory and, with dropout active, reports noisy numbers you'll chase for a day.

---

### Warmup skipped on restarts.

Resuming with a fresh scheduler re-runs warmup (harmless) or, worse, jumps to peak LR on a cold optimizer (not harmless). The schedule state is state (§4.3).

---

### Trusting `nvidia-smi` utilization.

It reports "a kernel was running," not "the machine was fed." MFU (Ch. 8) is the honest metric; the profiler is the honest witness.

## 4.10 Where this shows up later

This chapter is the book's parts list, and every later chapter picks up one part. The loop's FLOPs get priced in Chapters 6 and 8 (the 6ND rule, rooflines, MFU) and scheduled in Chapter 17. The sixteen bytes per parameter become Chapter 14's sharding problem (ZeRO/FSDP split exactly those five components across GPUs) and Chapter 15's compression problem (offload, 8-bit optimizers, activation checkpointing). The memory ladder of §4.5 reappears as the villain of Chapter 8's roofline and the whole design logic of Chapter 13's attention kernels. The interconnect of §4.6 is Chapter 14's subject entirely. The checkpoint discipline of §4.3 returns at scale in the case study of Chapter 29 — one interruption every three hours, remember — and the loader pathologies of §4.9 are why Chapter 10 treats data pipelines as systems engineering, not scripting. When any of those chapters feels abstract, come back to this one: it is all the same five lines of Python, meeting the same ladder of memory and wires.

## 4.11 Summary

Training is one short loop — load, forward, loss, backward, step — running on a machine whose defining constraints are memory and wires, not arithmetic. The learning rate that loop applies is shaped, not constant: warmup protects the early steps while Adam's per-parameter statistics form, and a cosine glide shrinks the stride as the model closes in. A checkpoint is the run's *entire* state — weights, optimizer moments, schedule, RNGs, loader position — sized at ~14 bytes per parameter on disk and scheduled by failure arithmetic, because at scale something breaks every few hours. The GPU is a matmul machine that is only fast when fed large regular multiplies, and it eats from a memory ladder — SRAM, HBM, host, storage — where each rung is roughly ten times slower and everything in Parts III–IV is a scheme for doing more math per trip down. Training state costs sixteen bytes per parameter (weights 2, gradients 2, optimizer 12) plus elastic activations — the anatomy that makes a 2 GB model a 41 GB tenant and a 70B model a 1.1 TB distributed system. Multi-GPU training adds the wire as a resource, syncing gradients every step across an interconnect ladder of its own. Keep the ownership map of the software stack, debug OOMs in cost order, feed the GPU from an offline-tokenized pipeline, and build determinism on purpose — the rest of this book is making each line of this chapter fast.

---

## References

1. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Reports 419 unexpected interruptions across the 54-day 405B pre-training window, ~78% hardware-related; describes checkpoint/restart operations at scale.
2. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*. arXiv:1412.6980. The moment estimates behind §4.2's warmup rationale and §4.7's 8 bytes.
3. Micikevicius, P., et al. (2018). Mixed Precision Training. *ICLR*. arXiv:1710.03740. The fp32-master-copy recipe that fixes §4.7's byte accounting.
4. Rajbhandari, S., Rasley, J., Ruwase, O., & He, Y. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. *SC20*. arXiv:1910.02054. Names and shards exactly the five state components of §4.7 — Chapter 14 picks it up.
5. NVIDIA (2023). H100 Tensor Core GPU Architecture whitepaper. Peak bf16 tensor throughput, HBM3 bandwidth, NVLink rates behind §§4.4–4.6's ladder numbers.

---

Next in Part I: *Chapter 5 — The lifecycle of an LLM*. This chapter's parts list is priced in *Chapters 6 and 8*, scheduled in *Chapter 17*, sharded in *Chapter 14*, and shrunk in *Chapter 15*. · [Training Efficient LLMs](#)